SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Technical Presentation Questions

Course Code:	CSA0315	Course Title:	Data Structures
CO Assessed:	CO1	Assessment:	Assessment Tool 2 – Technical Presentations Questions
Weightage:	10% of CIA	Total Marks:	100
CO1 Statement:	Basics, Data, Data object, Data structure, Abstract Data Types (ADT), realization of ADT in 'C', Primitive and non-primitive data structure, Linear and non-linear data structure Arrays & Recursion, Performance analysis and Measurement: Time and space analysis of algorithms, Average, best and worst-case analysis.		
Student Name:	K.B.Tharunika	Reg. No.:	192525276
Section / Batch:	192525276	Date:	

Code of Conduct

I K.B.Tharunika (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K.B.Tharunika

Instructions

- This test contains 3 Concept mapping questions. Total: 100 Marks.
- When a student answer using a concept map, in evaluation the answer should contain following four requirements: Understanding of concepts, Connections between ideas, Logical structure, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.
- Create a PPT with 15 slides – Laptop permitted for presentation in the class. Take Print out and submit in the class.

Section / Topic	Focus	Q Nos	Marks
Realization of ADT in 'C'- Array DS- Performance analysis and Measurement: Time and space analysis of algorithms.	Linked List data Structure	1	30
Primitive and non-primitive data structure, Linear and non-linear data structure Arrays	Tries data Structure	2	40
Dynamic linear Data Structures	Queue Data structures	3	40
		GRAND TOTAL:	100 Marks

SAVEETHA
SCHOOL OF ENGINEERING
Affliated to AICTE | IET-UK Accreditation

SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA0315– DATA STRUCTURES

Efficient Playlist Management Using Doubly Linked List

GUIDED BY
Dr.T.RajeshKumar

PRESENTED BY
K.B.Tharunika (192525276)
BTech AIML

INTRODUCTION

- A music playlist stores songs in sequence.
- A music playlist stores songs in sequence.
- Users can add, delete, move, or reorder songs anytime.
- The data structure should support dynamic updates efficiently.
- **Doubly Linked List** is best because each song is connected

to both previous and next songs.

- A **Doubly Linked List** is the most suitable data structure because it supports efficient insertion, deletion, and forward/backward navigation between songs.

Music Streaming Application

(Doubly Linked List)

OPERATIONS

- Add Song
- Delete Song
- Reorder Song
- Play Next Song
- Play Previous Song
- Display Playlist

Why Doubly Linked List?

- Easy insertion and deletion.
- Supports forward and backward navigation.
- Efficient song reordering.
- No need for contiguous memory.

Operation	Time Complexity
Insert	O(1)
Delete	O(1)
Traverse	O(n)
Search	O(n)

EXAMPLE

Before:

Song A ⟷ Song B ⟷ Song C

After adding Song D

Song A ⟷ Song B ⟷ Song C ⟷ Song D

APPLICATIONS

- Spotify
- Apple Music
- YouTube Music
- VLC Media Player
- Music Library Management

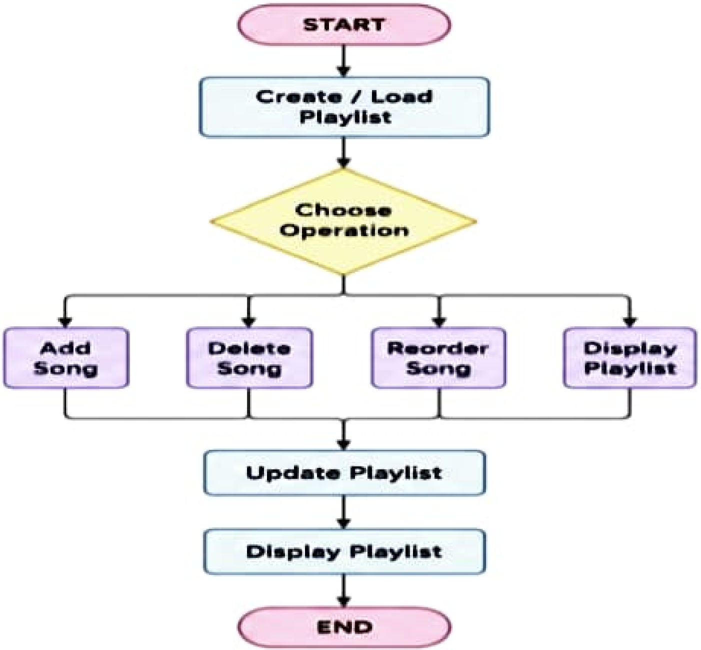
Spotify

Apple Music

YouTube Music

VLC Media Player

FLOWCHART


START
Create / Load Playlist
Choose Operation
Add Song
Delete Song
Reorder Song
Display Playlist
Update Playlist
Display Playlist
END

Conclusion

- Doubly Linked List provides flexible insertion and deletion.
- Easy forward and backward navigation.
- Suitable for dynamic playlists
- Improves the overall user experience in music streaming applications.
- Widely used in **Spotify, Apple Music, YouTube Music, and VLC Media Player**.
- Efficient memory management and smooth playlist operations.
- Therefore, a **Doubly Linked List** is the best choice for managing music playlists dynamically.

Q2. Efficient Dictionary Management Using Hash Table

- A dictionary application stores a large number of words and their meanings.
- Users need to search for words quickly and efficiently.
- A **Hash Table** is the most suitable data structure because it uses a **hash function** to store and retrieve words in constant time, making searches very fast.
- Hash Tables support fast insertion, deletion, and searching of words
- The average time complexity for search operations is O(1).

OPERATIONS

- Insert Word
- Search Word
- Delete Word
- Update Meaning
- Handle Collision

TIME COMPLEXITY

Operation	Time Complexity
Search	O(1)
Insert	O(1)
Delete	O(1)

EXAMPLE

Hash Table

Index	0	1	2	3	4	5	6	7
Word	-	-	Ball	-	Cat	Apple	-	Dog
Meaning	-	-	A round object	-	A small animal	A fruit	-	A pet animal

Search Operation (Example: "Apple")

Apple → **Hash Function** → **Index = 5** → **Found ✓**

"A Hash Table quickly finds a word by mapping it to a unique index using a hash function.

APPLICATIONS

- Dictionary Applications

Quickly search for word meanings

- Spell Checkers

Used to check whether the word exists in dictionary

- Search Engines

Hash tables are used for indexing and fast search

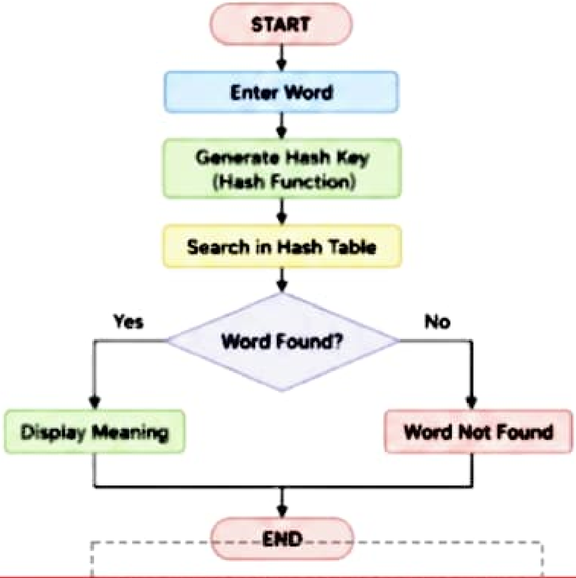
- Password Verification

Stores and verifies passwords effectively.

- Database Indexing

Speeds up data retrieval in databases

FLOW CHART


START
Enter Word
Generate Hash Key
(Hash Function)
Search in Hash Table
Word Found?
Yes
No
Display Meaning
Word Not Found
END

CONCLUSION

- In conclusion, a **Hash Table** is the most suitable data structure for a dictionary application because it provides **very fast word searching, insertion, and deletion**.
- By using a **hash function**, each word is stored at a specific index, allowing direct access without searching the entire database.
- Hash Tables are widely used in **online dictionaries, spell checkers, search engines, and database indexing systems** because of their speed and efficiency.
- Therefore, the Hash Table is an ideal choice for building a fast, reliable, and efficient dictionary application.

Topic 3: Priority-Based Task Scheduling Using a Priority Queue

INTRODUCTION:

- A task scheduling system manages multiple jobs based on their priority levels.
- High-priority tasks are executed before low-priority tasks to ensure efficient use of system resources.
- A **Priority Queue (Heap)** is the most suitable data structure because it allows quick insertion of tasks and efficient retrieval of the highest-priority task.

Task Scheduling System (Priority Queue)

Operations

- Insert Task
- Assign Priority
- Execute Highest-Priority Task
- Delete Completed Task
- Display Task Queue

TIME COMPLEXITY:

Operation	Time Complexity
Insert	O(log n)
Delete	O(log n)
Get Highest Priority	[illegible]

Example

Tasks with Priority:

Task	Priority	Description
T1	1	Emergency Task
T2	2	Print Document
T3	3	Email Notification
T4	4	Backup Files

Execution Order (High Priority to Low Priority):

T1 → T2 → T3 → T4

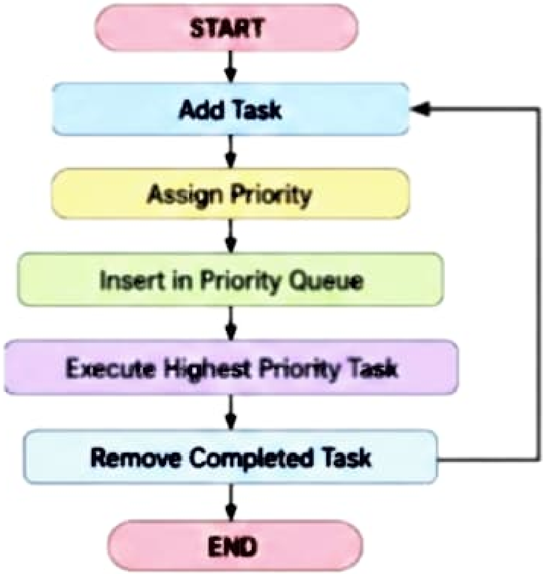
(Emergency → Print → Email → Backup)

"A Priority Queue executes tasks from the highest priority to the lowest priority."

APPLICATIONS

- CPU Scheduling
- Hospital Emergency Systems
- Printer Queue Management

FLOWCHART


START
Add Task
Assign Priority
Insert in Priority Queue
Execute Highest Priority Task
Remove Completed Task
END

CONCLUSION

- **Priority Queue (Heap)** is the most suitable data structure for task scheduling systems because it executes tasks according to their priority.
- The Priority Queue supports fast insertion and deletion operations with **O(log n)** complexity and provides immediate access to the highest-priority task.
- It is widely used in operating systems, printer management, hospital emergency systems, and network scheduling.
- Therefore, the **Priority Queue** is the best choice for efficient and reliable task scheduling.

THANK YOU

Annexure A – Technical Presentation

1) A music streaming application maintains a playlist where songs can be added, removed, or reordered dynamically. Recommend a suitable data structure and justify based on flexibility and efficiency. Prepare a technical Presentation

2) A dictionary application needs to quickly search whether a word exists in its database. Suggest an appropriate data structure and justify based on search performance. Prepare a technical Presentation.

3) A task scheduling system assigns jobs based on execution priority levels. Suggest a suitable data structure and justify performance efficiency.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Faculty In-charge	Course Coordinator	Program Director
Signature: ______________	Signature: ______________	Signature: ______________
Date: ______________	Date: ______________	Date: ______________